

Checkers – Java

Rapport final

RANDRIAMIHAJA Lalatiana
RAMANANAHERISOA Fy-Tommy
HOAREAU Faniry
ATWI Daniel
RAZAFINDRAKOTO Sarah

8 avril 2026

Introduction

1 Contexte et objectifs du projet

1.1 Historique

Le jeu de dames est l'un des plus anciens jeux de plateau stratégiques, dont les origines remontent à plusieurs millénaires dans les civilisations méditerranéennes. Perfectionné au fil des siècles, il a acquis ses règles modernes en Europe au cours du Moyen-Âge et fait aujourd'hui l'objet de compétitions internationales officielles régies par la Fédération Mondiale du Jeu de Dames (FMJD).

1.2 Présentation du jeu

Le jeu de dames est un jeu de stratégie abstraite opposant deux joueurs sur un plateau quadrillé. Chaque joueur manœuvre ses pièces en diagonale dans le but de capturer l'intégralité des pièces adverses ou de les immobiliser. Sa simplicité d'accès contraste avec la profondeur tactique qu'il offre : la gestion des prises obligatoires, les enchaînements de captures multiples et la promotion des pions en dames confèrent au jeu une richesse stratégique appréciée aussi bien des joueurs occasionnels que des compétiteurs de haut niveau.

1.3 Objectifs du projet

Le projet a pour objectif de concevoir et développer un logiciel permettant de jouer au jeu de dames conformément aux règles officielles. Le programme doit permettre à deux joueurs de s'affronter sur un damier, soit en mode local, soit à distance via un réseau.

Le logiciel garantit le respect strict des règles du jeu tout en proposant une expérience utilisateur claire, robuste et extensible. Il s'inscrit dans une démarche de développement logiciel complète, intégrant la qualité du code, la documentation, les tests et les performances.

1.4 Variantes supportées

Le jeu de dames se décline en plusieurs variantes internationales, que le programme supporte via une option de configuration du plateau. Le style anglo-saxon, appelé Draughts, se joue sur un plateau de 8×8, chaque joueur disposant de 12 pions. Le jeu de dames international adopte quant à lui un plateau de 10×10, avec 20 pions par joueur. Enfin, les dames canadiennes étendent encore la surface de jeu avec un plateau de 12×12, portant le nombre de pions à 30 par joueur.

Les images suivantes ont été obtenues à l'aide de notre implémentation.

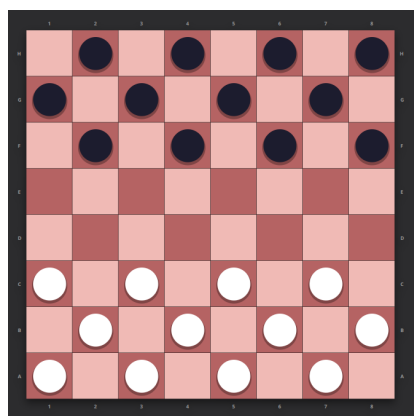


FIGURE 1 – Plateau 8×8

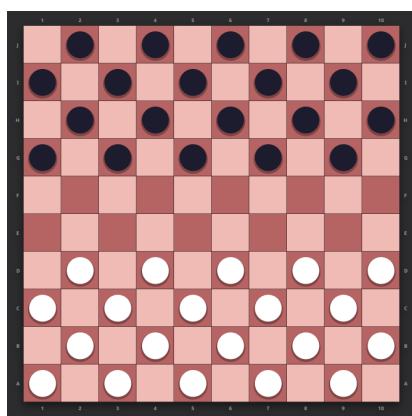


FIGURE 2 – Plateau 10×10

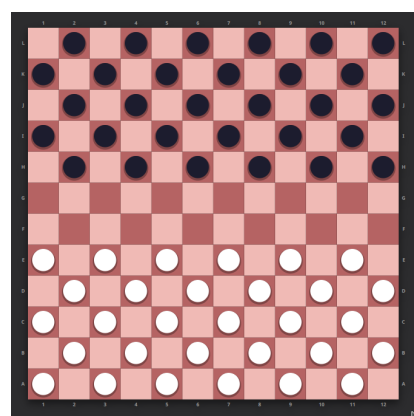


FIGURE 3 – Plateau 12×12

1.5 Règles du jeu

1.5.1 Mise en place et déroulement

Le damier est orienté avec une case foncée en bas à gauche. Les pions se placent exclusivement sur les cases foncées des quatre premières rangées de chaque camp. Les joueurs jouent chacun à leur tour ; les blancs commencent toujours.

1.5.2 Déplacements

Un pion se déplace d'une case vers l'avant en diagonale. Lorsqu'il atteint la dernière rangée adverse et s'y arrête, il est promu en dame, ce qui se matérialise visuellement par l'ajout d'un point jaune. La dame bénéficie alors d'une mobilité bien supérieure : elle peut parcourir librement une diagonale entière, d'autant de cases qu'elle le souhaite, aussi bien vers l'avant que vers l'arrière.

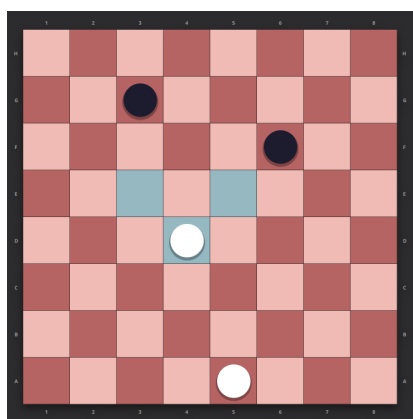


FIGURE 4 – Déplacement d'un pion

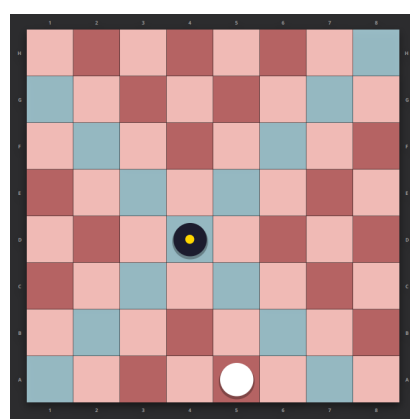


FIGURE 5 – Déplacement d'une dame

Les cases surlignées en bleu indiquent les déplacements possibles de la pièce sélectionnée.

1.5.3 La prise par un pion

Un pion capture un pion adverse en sautant par-dessus lui pour se poser sur la case vide située immédiatement derrière. La prise peut s'effectuer vers l'avant comme vers l'arrière. **La prise est obligatoire.** De plus, si après une capture le pion se retrouve à nouveau en position de prise, il doit continuer jusqu'à épuisement des captures possibles (rafle). Les pions capturés sont retirés du plateau uniquement à l'issue complète de la rafle, et non au fur et à mesure. La figure 6 illustre le mécanisme de prise simple, tandis que la figure 7 montre un exemple de rafle enchaînant plusieurs captures successives.

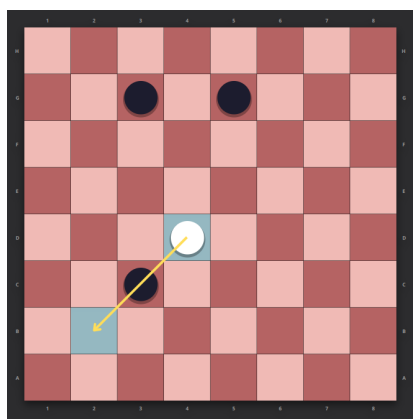


FIGURE 6 – Prise d'un pion

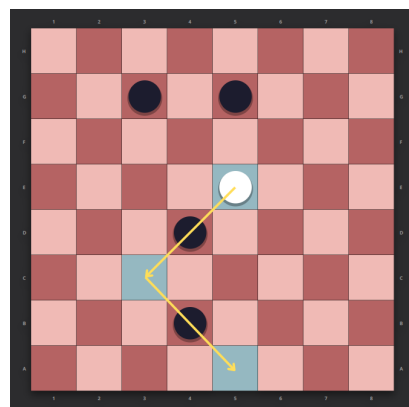


FIGURE 7 – Rafle

1.5.4 La prise majoritaire

Lorsque plusieurs séquences de capture sont possibles, le joueur est tenu de choisir celle qui capture le plus grand nombre de pièces, quel que soit leur rang. Ainsi, comme l'illustre la figure 10, une séquence permettant de prendre deux pions est prioritaire sur la prise d'une dame isolée.

1.5.5 La prise par la dame

Grâce à son amplitude de mouvement, la dame dispose de capacités de capture étendues. Elle doit prendre tout pion situé sur sa diagonale dès lors qu'une case libre est disponible immédiatement après. On ne peut passer qu'une seule fois sur un même pion, mais on peut traverser deux fois la même case. La figure 8 présente un exemple de prise par la dame, et la figure 9 illustre le cas particulier où la dame repasse sur une case déjà traversée au cours de sa rafle.

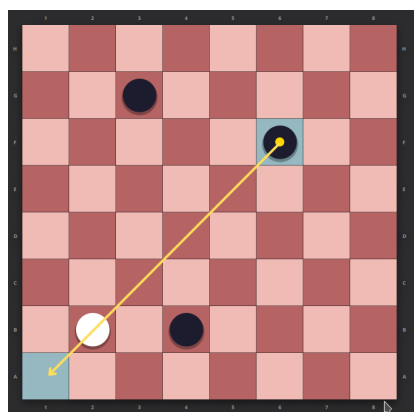


FIGURE 8 – Prise par la dame

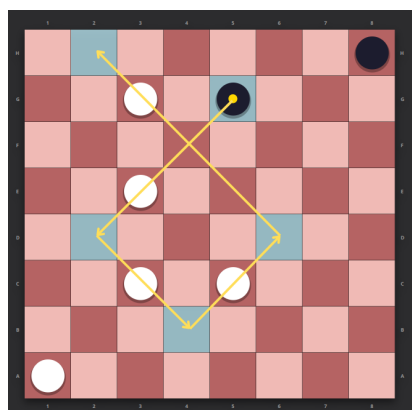


FIGURE 9 – Prise par la dame avec passage sur une même case

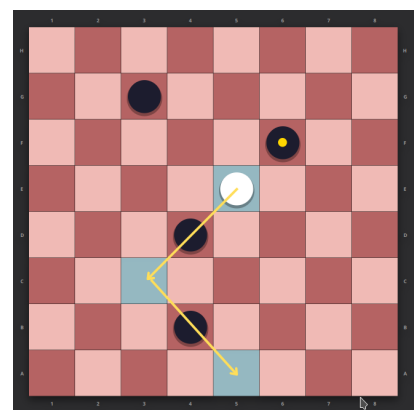


FIGURE 10 – Prise majoritaire : 2 pions vs 1 dame

1.5.6 Conditions de fin de partie

Une partie prend fin dans l'un des cas suivants :

Victoire par capture. Un joueur remporte la partie en capturant l'intégralité des pièces adverses.

Victoire par blocage. Si un joueur ne peut effectuer aucun déplacement alors qu'il lui reste des pièces, il perd immédiatement la partie.

Nulle par accord de position. La partie est déclarée nulle si la même position se reproduit trois fois au cours de la partie (règle de la répétition triple).

Nulle par absence de progrès. Si aucune capture ni aucun mouvement de pion n'est effectué pendant 25 tours consécutifs (soit 50 demi-coups), la partie est déclarée nulle.

Nulle en fin de partie. Lorsqu'un joueur ne dispose plus que d'une seule pièce, si aucune issue décisive n'est atteinte en 16 tours (soit 32 demi-coups), la partie est déclarée nulle faute de matériel suffisant.

Nulle par blocage mutuel. Dans le cas exceptionnel où aucun des deux joueurs ne peut effectuer de mouvement, la partie est également déclarée nulle.

1.6 Étude de l'existant

Dans le cadre de notre phase d'analyse, nous avons étudié deux projets open-source de jeu de dames afin d'identifier des approches architecturales et algorithmiques pertinentes pour notre développement.

AwsDame

AwsDame [2] est une plateforme de jeu de dames multijoueur en temps réel développée en JavaScript/Node.js, distribuée sous licence MIT. Bien que la pile technologique diffère de la nôtre (Java), son architecture réseau orientée événements offre des points de comparaison instructifs.

En particulier, ce projet adopte une logique de *matchmaking automatique* entre joueurs connectés : dès que deux joueurs sont disponibles, une session de jeu est lancée sans intervention manuelle. Contrairement à AwsDame qui s'appuie sur WebSockets, le cahier des charges de notre projet impose l'utilisation de sockets TCP, dont la mise en œuvre est détaillée dans la section 3.3.

Raven Checkers

Raven Checkers [1] est un programme de dames open-source développé en Python sous licence GPL, conçu à des fins pédagogiques pour l'étude des tactiques de jeu et des algorithmes d'intelligence artificielle. Il couple la fonction d'évaluation de *Simple Checkers* aux algorithmes de recherche du projet AIMA [3].

L'analyse de ce projet a principalement influencé notre réflexion sur les algorithmes d'IA. Raven Checkers propose un MinMax classique sans gestion du temps, ce qui le rend peu adapté à des contraintes de jeu réel. Cette lacune nous a conduits à étudier des approches plus robustes : approfondissement itératif, élagage alpha-bêta et Monte Carlo Tree Search (MCTS). Ces choix algorithmiques sont présentés en détail dans la section 2.

Bilan comparatif

Le tableau 1 synthétise les principales différences entre les deux projets analysés et notre propre implémentation.

TABLE 1 – Comparaison des projets existants et de notre implémentation

Critère	AwsDame	Raven Checkers	Notre projet
Langage	JavaScript	Python	Java
Transport réseau	WebSocket	—	TCP (sockets)
Matchmaking automatique	Oui	—	Oui
Suivi des scores	—	—	Oui (W/L/D)
MinMax	—	Oui	Oui
Élagage alpha-bêta	—	Non	Oui
Approfondissement itératif	—	Non	Oui
MCTS	—	Non	Oui
Contrainte de temps pour l'IA	—	Non	Oui (configurable)

L'étude de ces deux projets nous a permis de valider nos choix de conception et d'identifier les lacunes à combler : gestion du temps dans la recherche adversariale, richesse algorithmique de l'IA, et protocole réseau adapté à un client Java interactif.

2 Algorithme et Structure de données

2.1 Bitboard

Nous utilisons des bitboards pour représenter l'état du plateau. Cette approche permet de représenter l'état du plateau sous forme d'entiers primitifs et de calculer les coups de manière globale via des opérations bit à bit, offrant des performances bien supérieures aux matrices 2D classiques.

2.1.1 Représentation et Numérotation des cases

Sur les 100 cases d'un plateau standard (10×10), seules les 50 cases noires sont utilisées. Nous utilisons la **notation de Manoury** pour numéroter ces cases actives de 0 à 49, de bas en haut et de gauche à droite.

L'état complet du jeu est encodé sur quatre entiers de 64 bits (`long` en Java), où chaque bit correspond à une case active (1 si la case est occupée, 0 si elle est vide). Cette représentation permet de s'affranchir de la gestion itérative d'un tableau à deux dimensions. L'état du plateau n'est plus une collection d'objets ou d'entiers dispersés, mais un

vecteur binaire unique permettant d'évaluer la légalité des coups par de simples masquages logiques plutôt que par des parcours de boucles imbriquées.

	·	45	·	46	·	47	·	48	·	49
40	·	41	·	42	·	43	·	44	·	
	·	35	·	36	·	37	·	38	·	39
30	·	31	·	32	·	33	·	34	·	
	·	25	·	26	·	27	·	28	·	29
20	·	21	·	22	·	23	·	24	·	
	·	15	·	16	·	17	·	18	·	19
10	·	11	·	12	·	13	·	14	·	
	·	5	·	6	·	7	·	8	·	9
0	·	1	·	2	·	3	·	4	·	

2.1.2 Gain de mémoire chiffré

L'intérêt premier du bitboard est la compression des données. Comparons cette approche avec une structure de données "classique" comme un tableau à deux dimensions en Java (`int[10][10]`). Un tableau 2D classique nécessite 100 cases, chacune stockant un entier de 32 bits, soit un total de 3200 bits pour représenter l'état du plateau. En revanche, avec les bitboards, nous représentons l'ensemble du plateau avec seulement 4 variables de type `long` (64 bits chacune) : une pour les pions blancs, une pour les pions noirs, un pour les dames blanches et un pour les dames noires. Cela représente seulement 256 bits au total.

Le gain en mémoire vive est d'un facteur 12, ce qui accélère drastiquement les copies de plateaux lors de la recherche de l'IA.

2.1.3 Application des opérations binaires

L'avantage majeur du bitboard n'est pas seulement le stockage, mais la manipulation du plateau via le processeur. Au lieu de parcourir le plateau case par case, nous utilisons :

- **ET logique** ($\wedge$) : Pour détecter les collisions (ex : croiser un masque de déplacement avec le bitboard des pièces adverses).
- **OU logique** ($\vee$) : Pour fusionner les bitboards (ex : Blancs $\vee$ Noirs) et obtenir la carte de toutes les cases occupées.
- **NON logique** ($\neg$) : Pour inverser la carte d'occupation et isoler instantanément toutes les cases vides du damier.
- **Décalage binaire** ($\ll$, $\gg$) : Pour traduire toutes les pièces d'un joueur dans une direction donnée en une seule instruction d'horloge.

2.2 Algorithmes de manipulation

L'intérêt de la section précédente prend tout son sens ici : la génération des coups légaux repose sur la translation temporelle des bits.

TABLE 2 – Comparaison des décalages binaires selon la taille du plateau

Taille Plateau	8x8 (32 bits)		10x10 (50 bits)		12x12 (72 bits)	
Transition Ligne	P $\rightarrow$ I	I $\rightarrow$ P	P $\rightarrow$ I	I $\rightarrow$ P	P $\rightarrow$ I	I $\rightarrow$ P
Haut-Gauche	+3	+4	+4	+5	+5	+6
Haut-Droite	+4	+5	+5	+6	+6	+7
Bas-Gauche	-5	-4	-6	-5	-7	-6
Bas-Droite	-4	-3	-5	-4	-6	-5

Le tableau 2 synthétise les constantes de décalage utilisées par notre moteur de jeu. La manipulation de l'état du plateau via les bitboards repose sur une propriété géométrique fondamentale : un déplacement diagonal correspond à une translation d'index fixe, à la condition de prendre en compte la parité de la ligne d'origine (notée **P** pour Paire et **I** pour Impaire).

En effet, dans notre représentation compressée où seules les cases sombres sont indexées, le damier forme un motif en quinconce. Par conséquent, une pièce située sur une ligne paire n'a pas le même décalage numérique avec ses voisins diagonaux qu'une pièce située sur une ligne impaire.

Pour illustrer avec le plateau standard 10 $\times$ 10 (où chaque ligne contient 5 cases actives) : un déplacement vers le Haut-Gauche depuis une ligne paire (**P $\rightarrow$ I**) nécessite d'ajouter +4 à l'index binaire, tandis que le même mouvement depuis une ligne impaire (**I $\rightarrow$ P**) nécessite d'ajouter +5.

2.2.1 Déplacement et prise par un pion

Pour déterminer si un pion peut se déplacer, le moteur doit avant tout s'assurer que la case de destination est libre. Au lieu de maintenir une structure de données lourde et redondante pour pister les cases vides, notre implémentation calcule cette information à la volée. Nous fusionnons instantanément les quatre *bitboards* (les pions et dames de chaque joueur) au moyen d'une simple opération logique "OU" ($\vee$). Cette fusion crée une carte virtuelle regroupant toutes les pièces présentes sur le damier :

$$\text{Occupées} = \text{PionsBlancs} \vee \text{DamesBlanches} \vee \text{PionsNoirs} \vee \text{DamesNoires}$$

Grâce à ce masque global, la méthode `isOccupied(i)` de notre classe `Board` est capable de vérifier l'état de n'importe quelle case en temps constant.

La génération des mouvements d'un pion est ensuite confiée à la méthode `pawnSimpleTargets(...)`. Son fonctionnement est très intuitif : elle teste les diagonales adjacentes à la pièce, écarte naturellement les directions illégales (comme le fait d'avancer vers le bas pour un pion blanc), puis valide les trajectoires dont la case d'arrivée n'est pas marquée dans notre masque des cases occupées.

Exemple concret de prise : Pour qu'une capture soit possible, un pion doit sauter par-dessus un pion adverse et atterrir sur une case vide. Sur un plateau 10×10 , supposons un pion blanc en case 32 (ligne paire). S'il veut capturer un pion noir situé en haut à gauche (case 27, ligne impaire), le décalage pour atteindre le pion noir est de +5. Pour atterrir derrière ce pion, le saut continue vers la case 21 (ligne paire), nécessitant un nouveau décalage de +6.

Le décalage total pour cette prise est donc de $5 + 6 = 11$ cases indexées. La validation logique peut se lire ainsi :

$$\text{CibleSaut} = (\text{PionBlanc} \gg 5) \wedge \text{PionNoir}$$

$$\text{AtterrissageValide} = (\text{CibleSaut} \gg 6) \wedge \neg(\text{Occupées})$$

Dans le code, les prises de pion sont générées par `getBestPawnCaptures(...)` et `pawnMultiCaptures(...)` (DFS). Le coût structurel d'un test local reste en $O(1)$, mais l'exploration de toutes les rafles possibles est exponentielle dans le pire cas : $O(B^k)$, avec B le facteur de branchement des captures et k la profondeur maximale de rafle.

Autres algorithmes bitboard présents dans le code. Les deux algorithmes présentés ici ne sont pas les seuls. Le module `Board` utilise également des opérations bitboard pour :

- la génération des coups simples de dame (`checkerSimpleTargets`);
- la génération des prises multiples de dame par DFS (`checkerMultiCaptures`);
- la sélection des coups légaux avec priorité aux prises majoritaires (`getValidMoves`, `getBestPawnCaptures`, `getBestCheckerCaptures`);
- l'application d'un coup, la suppression des pièces capturées et la promotion (`applyMove`, `promoteBit`).

2.2.2 Déplacement et prise par une dame

Contrairement au pion, la dame peut parcourir un nombre illimité de cases vides sur une diagonale. D'un point de vue binaire, un simple décalage ne suffit pas : il faut propager le bit sur toute la diagonale jusqu'à rencontrer un obstacle. C'est la technique du *sliding piece generation*.

Voici le pseudo-code illustrant la génération des coups possibles pour une dame :

```
Fonction GenererCoupsDame(bitboard_dame, direction):
    coups_possibles = 0
    // Premier decalage dans la direction souhaitee
    rayons = decaler(bitboard_dame, direction)

    // Tant qu'on se deplace sur des cases vides
    Tant que rayons != 0:
        // On valide les cases vides rencontrees comme coups possibles
        coups_possibles = coups_possibles OU (rayons ET cases_vides)

        // On continue la propagation uniquement depuis les cases vides
        // La propagation s'arrete si on touche une piece ou le bord
        rayons = decaler(rayons ET cases_vides, direction)

    Retourner coups_possibles
```

2.3 Complexité du Board

2.3.1 Hypothèses et notations

Dans cette partie, nous analysons la classe `Board`, qui constitue le moteur de génération et d'application des coups. On note n le nombre de cases actives (32, 50 ou 72 selon la variante), p le nombre de pièces présentes sur le plateau, et c le nombre de pièces capturées par un coup donné. Les opérations bit-à-bit de base (tests, OR, AND, décalages, `bitCount`) sont considérées en temps constant sur mot machine.

2.3.2 Accès, comptage et conversion

Les requêtes élémentaires de présence, comme `isBitWhitePawn(int)`, `isBitBlackPawn(int)`, `isBitWhiteChecker(int)` et `isBitBlackChecker(int)`, reposent sur un test de bit unique ; elles sont donc en $O(1)$. Les vérifications de bord `onLeftEdge(int)` et `onRightEdge(int)` sont également en $O(1)$.

Les conversions d'indices (`indexToSquare(int)` et `boardToBitIndex(int, int)`) sont arithmétiques et valent $O(1)$. La méthode `squareToIndex(String)` reste $O(1)$ du point de vue asymptotique, car la longueur des coordonnées est bornée par la taille du plateau supportée (8, 10, 12).

Les compteurs `whitePawnsCount()`, `blackPawnsCount()`, `whiteCheckersCount()` et `blackCheckersCount()` utilisent `Long.bitCount` sur deux mots 64 bits ; ils s'exécutent donc en $O(1)$.

2.3.3 Application d'un coup et mutations locales

La méthode `applyMove(Move)` comporte trois étapes : déplacement de la pièce, suppression des captures, et promotion éventuelle. Le déplacement et la promotion sont en $O(1)$, tandis que la suppression des captures parcourt la liste `move.getCaptured()` ; le coût est donc $O(c)$. On obtient ainsi :

$$T(\text{applyMove}) = O(1 + c) = O(c).$$

Les helpers de mutation ciblée (`addWhitePawn`, `removeWhitePawn`, `addBlackChecker`, etc.) ne modifient qu'un bit et restent en $O(1)$. Les méthodes utilitaires de test (`restorePiece`, `demoteBit`, `remove`, `addWhite`, `addBlack`, `clearBoard`) sont donc toutes en $O(1)$ dans ce modèle borné.

2.3.4 Génération des coups

La génération globale `getValidMoves(boolean)` effectue un balayage des cases actives puis appelle des routines de coups simples et de prises. Le coût structurel de ce balayage est $O(n)$, mais le coût dominant provient des DFS de captures : `dfsPawn(...)` et `dfsChecker(...)`.

Pour les coups simples, `pawnSimpleTargets(int, boolean)` teste un nombre constant de directions et reste en $O(1)$. `checkerSimpleTargets(int)` parcourt des rayons diagonaux, donc $O(n)$ dans le pire cas (plateau très dégagé).

Pour les prises, `pawnMultiCaptures(...)` et `checkerMultiCaptures(...)` explorent un arbre de séquences de capture. Leur pire cas est exponentiel en profondeur de capture k (en pratique bornée par le plateau), ce que l'on note $O(B^k)$, où B est le facteur de branchement effectif des continuations de prise. Par conséquent, la complexité de `getWhiteValidMoves()` / `getBlackValidMoves()` s'écrit :

$$O(n + \text{coût DFS captures}),$$

et est dominée, dans les positions tactiques, par l'exploration des chaînes de captures.

2.3.5 Copie de plateau

La méthode `copy()` alloue un nouvel objet puis recopie un nombre constant de champs primitifs (huit bitboards + métadonnées). Elle est donc en $O(1)$, ce qui est un point clé pour les algorithmes de recherche.

2.3.6 Synthèse Board

TABLE 3 – Complexité des principales méthodes de `Board`

Famille	Méthode	Complexité	Justification
Accès bitboard	<code>isBit*</code> , <code>onLeftEdge</code> , <code>onRightEdge</code>	$O(1)$	Un nombre constant de tests/logiques binaires sur un ou deux mots 64 bits.
Conversion	<code>indexToSquare</code> , <code>boardToBitIndex</code> , <code>squareToIndex</code>	$O(1)$	Calculs arithmétiques et parsing de coordonnées de taille bornée.
Mutation locale	<code>add*</code> , <code>remove*</code> , <code>promoteBit</code> , <code>demoteBit</code> , <code>restorePiece</code>	$O(1)$	Modification d'un nombre constant de bits/champs.

Suite à la page suivante...

Famille	Méthode	Complexité	Justification
Application d'un coup	<code>applyMove (Move)</code>	$O(c)$	Parcours des pièces capturées (taille c), le reste en temps constant.
Coups simples pion	<code>pawnSimpleTargets</code>	$O(1)$	Nombre fixe de directions testées, sans parcours global.
Coups simples dame	<code>checkerSimpleTargets</code>	$O(n)$ (pire cas)	Exploration potentielle de rayons diagonaux longs sur plateau dégagé.
Prises pion/dame	<code>pawnMultiCaptures</code> , <code>checkerMultiCaptures</code> , <code>dfsPawn</code> , <code>dfsChecker</code>	$O(B^k)$ (pire cas)	Exploration récursive d'un arbre de séquences de capture.
Génération globale	<code>getWhiteValidMoves</code> , <code>getBlackValidMoves</code>	$O(n + \text{DFS captures})$	Balayage des cases + génération des prises (coût dominant selon la position).
Copie	<code>copy</code>	$O(1)$	Recopie d'un nombre constant de champs primitifs.

3 Architecture du projet

3.1 Le Moteur de Jeu

L'architecture de notre application repose sur le patron Modèle-Vue-Contrôleur (MVC). Cette section détaille la conception de notre moteur de jeu (qui incarne le Modèle) et la façon dont il interagit avec le reste du programme pour assurer le bon déroulement d'une partie.

3.1.1 Rôle et responsabilités

Le moteur de jeu, regroupé dans le package `model`, a pour unique mission de simuler et de faire tourner la partie de dames. Il se concentre purement sur la logique mathématique et les règles officielles : il ne gère absolument pas les clics de souris, la saisie clavier ou l'affichage graphique.

Ses responsabilités sont réparties entre deux classes majeures. D'un côté, la classe `Board` agit comme le moteur physique du damier : elle manipule les pièces, valide si un coup respecte la géométrie du plateau et applique les règles de prises obligatoires. De l'autre, la classe `GameCheckers` incarne l'arbitre de la partie. C'est elle qui maintient l'état global du système. Elle garde la trace du joueur actif, stocke l'historique des mouvements pour permettre les retours en arrière (undo/redo), et détecte les conditions de fin de partie (victoire, blocage, ou match nul par répétition).

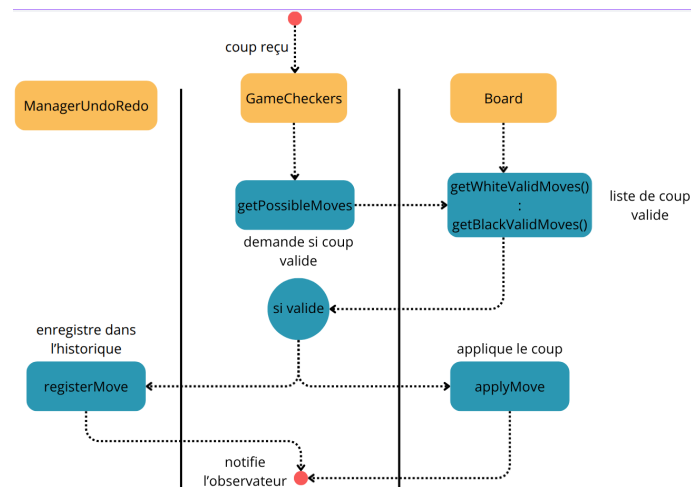


FIGURE 11 – Architecture du Modèle

3.1.2 Interface de communication (API)

Pour que le reste de l'application puisse interagir avec ce moteur, `GameCheckers` expose une interface de communication claire basée sur des actions d'entrée et des notifications de sortie.

En entrée, l'interface graphique ou textuelle ne modifie jamais le plateau directement. Elle passe par le `GameController` qui interroge d'abord le moteur via `getPossibleMoves(player)` pour s'assurer qu'une action est légale, puis lui demande de l'exécuter avec la méthode `applyMove(from, to)`.

En sortie, le moteur dicte le rythme grâce au patron de conception Observateur. Au lieu de forcer l'interface graphique à vérifier constamment si quelque chose a changé, le moteur travaille en silence et émet un signal de mise à jour via `notifyObservers()` uniquement lorsqu'un mouvement valide vient de modifier l'état du jeu.

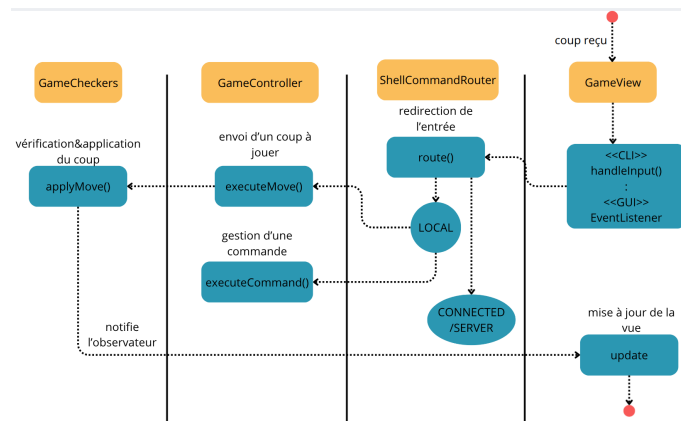


FIGURE 12 – Interaction entre le moteur et la vue via le contrôleur

Note : La bulle « CONNECTED/SERVER » visible sur ce schéma représente le point d'entrée vers les fonctionnalités réseaux. Le fonctionnement détaillé de ce module (protocole TCP, sockets et gestion du serveur) est traité spécifiquement dans la section 3.3.

3.1.3 Choix techniques et boucle de contrôle

D'un point de vue technique, la représentation du plateau ne repose pas sur un simple tableau à deux dimensions, mais sur des *bitboards* (comme détaillé dans la section algorithmique 2). Ce choix garantit un accès et une manipulation en temps $O(1)$ via des opérations binaires.

Concernant la dynamique, le jeu s'exécute au tour par tour, géré par une machine à états simple définie par l'énumération *State* (IN_GAME, PAUSE, FINISHED). L'évolution du système lors de l'application d'un coup suit toujours la même séquence logique stricte :

```
// Extrait simplifié de GameCheckers.java : évolution de l'état du système
public void applyMove(String from, String to, boolean isManoury) {
    // 1. Délégation de la physique au plateau
    board.applyMove(move);

    // 2. Mise à jour de l'état global (historique et tour de jeu)
    managerUndoRedo.registerMove(currentColor, move);
    this.isWhiteTurn = !this.isWhiteTurn;

    // 3. Signal de fin d'opération à l'interface
    notifyObservers();
}
```

Une fois le coup appliqué et la vue rafraîchie, le *GameController* vérifie si la partie est terminée. Si le jeu continue, il oriente le flux vers une attente de saisie utilisateur, ou lance immédiatement la réflexion de l'algorithme si le joueur suivant est une intelligence artificielle.

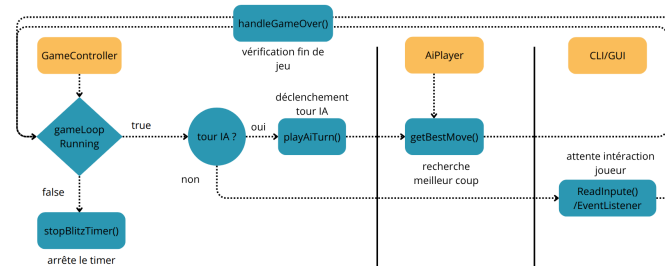


FIGURE 13 – Boucle de contrôle du jeu

3.2 Module affichage

Le module d’affichage correspond à la couche Vue de notre architecture MVC. Dans la pratique, il repose sur une base commune, `GameView`, puis sur trois implémentations adaptées au contexte d’exécution : `CommandLineInterface` pour le terminal, `GraphicalUserInterface` pour JavaFX, et `HeadlessView` côté serveur lorsque l’on veut exécuter une partie sans affichage. Ce découpage a été choisi très tôt pour éviter de mélanger logique d’interaction, rendu visuel et règles du jeu.

Le point important est que le contrôleur ne connaît que l’abstraction `GameView`. Quand un joueur agit, la vue transforme cette action en appel contrôleur (commande ou déplacement), puis le moteur met à jour l’état de jeu. La vue n’écrit jamais directement dans le modèle. Une fois le coup traité, le mécanisme d’observation notifie les vues, qui se contentent de se rafraîchir. Ce fonctionnement paraît simple, mais c’est ce qui nous a permis de garder exactement les mêmes règles en CLI, en GUI et en mode headless.

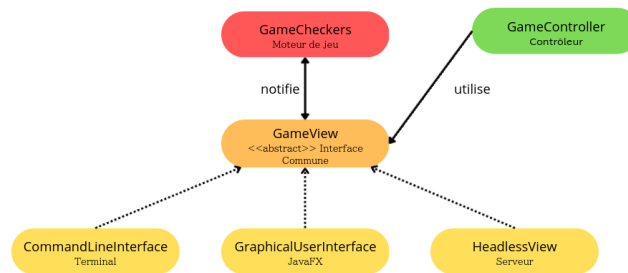


FIGURE 14 – Architecture des modules Model-View

En Interface en ligne de commande (CLI), nous avons utilisé `JLine` pour ne pas nous limiter à un `Scanner` minimaliste. On bénéficie ainsi d’une saisie plus robuste (édition de ligne, complétion, gestion propre des interruptions) et d’un comportement plus stable pendant les tests. Nous avons aussi séparé volontairement la lecture d’entrée du routage des commandes : `CommandLineInterface` lit et affiche, tandis que `ShellCommandRouter` décide quoi faire de la commande. Cette séparation nous a évité une classe trop chargée et a rendu les tests de parsing beaucoup plus simples.

En Interface graphique utilisateur (GUI), la même logique a été conservée, avec une organisation en sous-vues (`MainView`, `PlayView`, `BoardView`, `LogView`) pour garder du code lisible. Le principal choix technique ici concerne les threads : une mise à jour visuelle doit toujours passer par `Platform.runLater(...)`. Sans cela, les notifications venant du timer blitz ou d’un calcul d’IA peuvent provoquer des erreurs JavaFX difficiles à reproduire. Nous avons également lié la taille des cases à la taille de la fenêtre via des bindings, pour conserver un damier correct sur les trois formats de plateau pris en charge.

Avec le recul, ce module fonctionne surtout parce qu’il reste discipliné : l’affichage reste un adaptateur, pas un second moteur de jeu. C’est ce qui nous permet d’ajouter des fonctionnalités d’interface sans toucher aux règles, et d’exécuter les mêmes scénarios de jeu dans des environnements très différents.

3.2.1 Architecture interne de la GUI

L’implémentation de la GUI repose sur `javafx` et une décomposition en plusieurs composants spécialisés :

GraphicalUserInterface et le pattern Observer La classe `GraphicalUserInterface` étend `GameView`, ce qui lui permet de s’intégrer directement au pattern Observer déjà utilisé par le projet. Quand le modèle change, la GUI reçoit l’information via la méthode `update(GameCheckers)` et redirige la mise à jour vers les composants visuels.

Le point central ici est la gestion du thread JavaFX. Comme JavaFX impose que toute modification de l’interface passe par son thread dédié (`Application Thread`), les mises à jour sont dispatchées via `Platform.runLater(...)` :

```
@Override
public void update(GameCheckers game) {
    Platform.runLater(() -> {
        if (mainView != null) {
            mainView.update(game);
        }
    });
}
```

Ce pattern de thread est **central** : sans lui, toute mise à jour du modèle provenant d'un autre thread provoque une exception `IllegalStateException`. La classe gère aussi l'initialisation de la fenêtre principale, l'application de la feuille de style CSS et la fermeture contrôlée avec vérification des sauvegardes avant quitter.

MainView et organisation visuelle La fenêtre principale est construite par `MainView`, qui hérite de `BorderPane`. Ce choix permet de séparer proprement l'interface en trois zones : le menu en haut, le jeu au centre et la barre d'actions en bas.

`MainView` ne redessine pas le plateau lui-même ; ce rôle est délégué à `PlayView`, ce qui évite de concentrer la logique d'affichage dans une seule classe. Cette délégation renforce la maintenabilité et facilite les tests unitaires de chaque composant.

BoardView et rendu interactif `BoardView` est le composant qui affiche réellement le damier, les pièces, les sélections, les coups suggérés et les mouvements possibles. Le composant repose sur un `GridPane`, reconstruit visuellement à chaque `redraw`. L'interaction suit un schéma à deux clics : au premier clic, la pièce est sélectionnée et surlignée en bleu ; au second clic, la destination est envoyée au contrôleur pour exécution du coup.

Un point crucial est que la GUI **ne recalcule pas** les règles de dames. Elle demande directement au modèle quels coups sont légaux, puis ne garde que ceux qui concernent la pièce sélectionnée :

```
List<Move> possibleMoves = game.getPossibleMoves(game.getCurrentPlayer());

for (Move m : possibleMoves) {
    if (m.getFrom() == selectedIndex) {
        validDestinationSquares.add(board.indexToSquare(m.getTo()));
    }
}
```

L'interface ne décide pas ce qui est légal ou non ; elle se contente d'afficher proprement ce que le moteur autorise. Ce choix architectural évite de dupliquer la logique métier dans la vue et garantit la cohérence globale : une règle modifiée dans le moteur s'applique immédiatement sans changement de code GUI.

Le drag and drop a également été implémenté. Quand l'utilisateur commence à faire glisser une pièce, la case de départ est stockée dans le `Dragboard` JavaFX. Au dépôt, la vue reconstruit simplement le déplacement et l'envoi au contrôleur sans validation locale. Encore une fois, la vue gère l'interaction (UX), mais pas les règles.

ConfigDialog et interactions complexes Le dialogue de configuration `ConfigDialog` va au-delà d'une simple présentation de cases à cocher. Les composants réagissent entre eux : si le mode blitz est désactivé, le spinner de temps l'est aussi. Si aucune IA n'est active, les options liées aux IA sont désactivées automatiquement. Ces dépendances sont gérées via des listeners JavaFX sur les propriétés des contrôles.

Plus important encore, le dialogue reconstruit un objet `Configuration` exploitable directement par le moteur. Il ne renvoie pas juste des valeurs d'interface brutes : il normalise les choix, convertit les unités (par exemple, les secondes en millisecondes pour l'IA) et prépare les bons formats.

Tests de la GUI Exemple : `ConfigDialog`. Les tests utilisent JUnit 5, TestFX (framework pour tester JavaFX) et Mockito (pour simuler le contrôleur). Les composants JavaFX sont créés sur le bon thread à l'aide de `Platform.runLater(...)`, puis les interactions utilisateur sont simulées avec `FxRobot`.

Par exemple, un test vérifie que cocher ou décocher le mode blitz active ou désactive correctement le spinner de temps :

```
robot.interact(() -> blitzCheck.setSelected(false));
assertTrue(timeSpinner.isDisabled());

robot.interact(() -> blitzCheck.setSelected(true));
assertFalse(timeSpinner.isDisabled());
```

Cette approche garantit que le comportement du dialogue reste stable lors des refactorisations futures et valide que les interactions entre composants restent cohérentes.

3.3 Module Réseau

La couche réseau de l'application s'appuie sur une architecture client-serveur TCP, basant principalement sur le package `fr.ubordeaux.pdp.server`. La figure 15 en présente les composants principaux ainsi que leurs interactions.

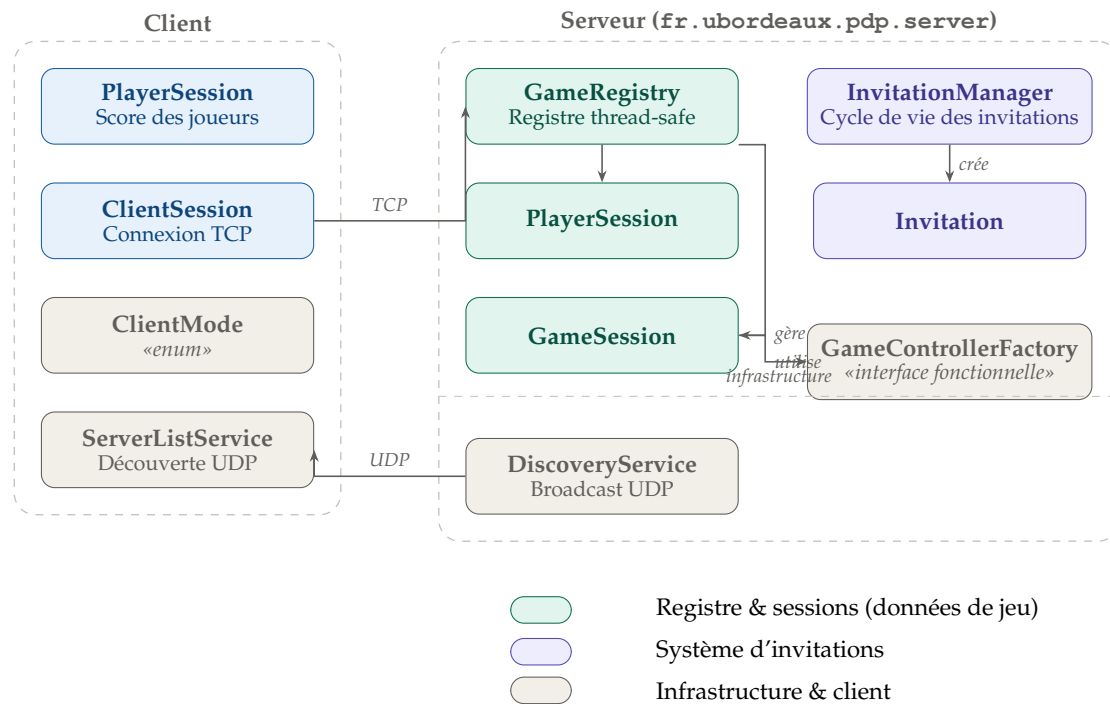


FIGURE 15 – Architecture structurelle de la couche réseau

Côté client. Le `ShellCommandRouter` constitue le point d'entrée unique pour toute saisie utilisateur. Il consulte l'énumération `ClientMode` (`LOCAL`, `CONNECTED`, `SERVER`) pour aiguiller chaque commande vers le bon gestionnaire, évitant ainsi les états incohérents (par exemple, démarrer un serveur alors qu'une connexion est déjà ouverte). La `ClientSession` encapsule la socket TCP, le fil d'écoute (*listener thread*) et le protocole de *handshake* (`REGISTER` / `WELCOME`). Le `ServerListService` écoute les annonces UDP diffusées par `DiscoveryService` afin de présenter la liste des serveurs disponibles sur le réseau local.

Côté serveur. Le `GameRegistry` centralise, de façon thread-safe, l'ensemble des `PlayerSession` et des `GameSession` actives. Il constitue la source de vérité pour l'état de chaque joueur connecté. L'`InvitationManager` orchestre le cycle de vie complet des invitations — de leur création jusqu'à l'acceptation, au refus ou à l'expiration automatique par un fil démon (*sweeper*) — en s'appuyant sur les objets `Invitation` immuables. Enfin, la `GameControllerFactory`, interface fonctionnelle, permet d'instancier un contrôleur de jeu isolé pour chaque session, garantissant l'indépendance des états de parties concurrentes.

3.4 Module Intelligence Artificielle

Le module IA repose sur une séparation nette entre trois responsabilités. D'abord, la classe abstraite `Ai` fournit l'ossature commune : elle valide les paramètres, crée une copie du plateau de recherche, instancie un `ManagerUndoRedo` dédié et délègue le calcul du meilleur coup aux algorithmes concrets. Ensuite, `GameCheckers` agit comme point d'assemblage en appelant `Ai.buildAi(cfg, ...)` et `Ai.buildEvaluator(cfg)` à partir de la configuration courante. Enfin, `AiPlayer` encapsule l'algorithme et l'évaluateur pour exposer une seule méthode `getBestMove(...)`, ce qui évite de disperser la logique de décision dans le reste du moteur.

Les classes d'évaluation `SimpleEvaluator`, `AdvancedEvaluator` et `MaxEvaluator` implémentent la même interface `Evaluator` mais avec des heuristiques de coût croissant. Elles sont consommées par les algorithmes de recherche pour donner une valeur aux feuilles de l'arbre. De son côté, `Mcts` n'utilise pas cette heuristique dans sa sélection principale, mais il reste compatible avec la même signature afin de pouvoir être instancié par le même pipeline que les autres IA.

3.4.1 SimpleEvaluator

`SimpleEvaluator` est l'heuristique la plus légère du module. Son implémentation parcourt toutes les cases actives du plateau via `board.getIndexMax()` et interroge directement les bitboards avec `isBitWhitePawn`, `isBitBlackPawn`, `isBitWhiteChecker` et `isBitBlackChecker`. Chaque pion vaut 100 points et chaque dame 350 points ; le score final est positif si la position favorise les blancs et négatif si elle favorise les noirs.

Cette classe n'a pas de dépendance métier lourde : elle ne consulte ni l'historique, ni les coups légaux, ni le système d'annulation. Elle sert donc de base de comparaison pour les recherches MinMax et comme option de configuration

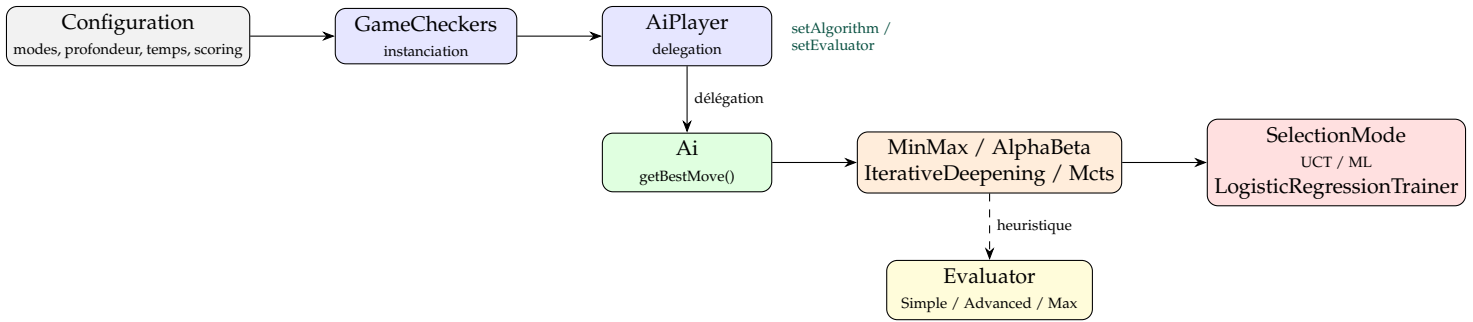


FIGURE 16 – Chaîne de dépendance du module Intelligence Artificielle

quand on veut une évaluation rapide et déterministe.

3.4.2 AdvancedEvaluator

`AdvancedEvaluator` reprend la logique matérielle de `SimpleEvaluator`, mais ajoute un bonus de progression pour les pions. Pour calculer ce bonus, la classe reconstruit la ligne de chaque case à partir de l'indice courant et de la taille du plateau, puis ajoute ou retire un bonus d'avancement proportionnel à la distance parcourue vers la promotion. Les dames gardent leur valeur fixe, ce qui permet de conserver une heuristique lisible tout en intégrant une notion de dynamique de partie.

En pratique, cette classe est utile pour les variantes où l'on veut un `MinMax` un peu plus sensible au développement des pions sans payer le coût des calculs de mobilité. Elle reste autonome vis-à-vis des autres classes du moteur et ne dépend que de `Board` pour récupérer la géométrie du plateau.

3.4.3 MaxEvaluator

`MaxEvaluator` est l'heuristique la plus coûteuse du trio. Elle combine d'abord le comptage matériel et le bonus d'avancement, puis ajoute deux critères stratégiques supplémentaires : le contrôle du centre et la mobilité. Le centre est détecté par la méthode privée `isInCenter(...)` à partir des coordonnées ligne/colonne, tandis que la mobilité est évaluée en appelant `board.getWhiteValidMoves()` et `board.getBlackValidMoves()`.

Cette classe introduit donc une dépendance forte vers le moteur de génération de coups de `Board`. Contrairement aux deux autres évaluateurs, elle peut déclencher une exploration de coups complète lors du calcul du score, ce qui la rend plus informative mais aussi plus lente. C'est précisément pour cela qu'elle est utilisée comme heuristique "maximale" dans la configuration `MinMax`.

3.4.4 MinMax

`MinMax` implémente la stratégie classique de recherche adversariale. Le point d'entrée réel passe par `Ai.getBestMove(...)`, qui travaille sur une copie du plateau et crée un `ManagerUndoRedo` dédié à l'exploration. L'algorithme parcourt ensuite l'arbre de jeu en alternant les fonctions `minMax(...)` et `maxMin(...)` : chaque coup est appliqué avec `board.applyMove(move)`, enregistré dans `undo.registerMove(...)` puis annulé immédiatement après l'évaluation de la branche.

La classe dépend donc à la fois de `Board` pour générer les coups légaux, de `ManagerUndoRedo` pour revenir en arrière, et de `Evaluator` pour scorer les feuilles. La gestion du temps repose sur une exception interne `TimeExceededException` qui interrompt proprement la récursion si le budget est dépassé. Le résultat est ensuite renvoyé sous forme d'un `Move` détaché de l'arbre de recherche afin de ne pas propager d'état interne vers la partie réelle.

3.4.5 MinMaxAlphaBeta

`MinMaxAlphaBeta` reprend exactement le même contrat que `MinMax`, mais ajoute deux bornes locales, α et β , dans les méthodes `maxValue(...)` et `minValue(...)`. Après chaque enfant évalué, la borne adaptée est mise à jour ; dès que $\beta \leq \alpha$, la boucle est interrompue et les frères restants sont ignorés. La qualité du résultat reste identique à celle de `MinMax`, mais l'arbre visité peut être très réduit lorsque l'ordre des coups est favorable.

Du point de vue des dépendances, la classe reste branchée sur les mêmes briques que `MinMax` : `Board`, `ManagerUndoRedo`, `Evaluator` et `PlayerColor`. La différence est purement algorithmique, ce qui permet à `GameCheckers` de la substituer à `MinMax` sans changer le reste du moteur.

3.4.6 IterativeDeepening

`IterativeDeepening` n'implémente pas son propre arbre de recherche ; elle encapsule un `MinMax` interne et relance le calcul pour des profondeurs croissantes, de 1 jusqu'à `maxDepth`. À chaque itération terminée, le meilleur

coup courant remplace le précédent. Si le temps expire au milieu d'une profondeur, la classe conserve simplement le dernier coup entièrement validé.

Cette organisation fait de la classe un adaptateur temporel plutôt qu'un nouvel algorithme de décision. Sa dépendance principale est donc le `MinMax` embarqué, auquel elle délègue toute l'exploration. Le bénéfice est double : on garantit toujours une réponse jouable et on améliore progressivement la qualité de la décision tant que le budget temps le permet.

3.4.7 Relations entre les classes

La figure 16 résume le chemin suivi lors d'une décision. `GameCheckers` choisit l'algorithme et l'évaluateur à partir de la configuration, `AIPlayer` les encapsule, puis `AI` orchestre la recherche sur une copie du plateau. Les algorithmes `MinMax` dérivés consomment ensuite l'évaluateur choisi, tandis que `Mcts` conserve la même interface mais s'appuie sur `SelectionMode` pour basculer entre UCT et la sélection guidée par apprentissage automatique.

Le cas MCTS introduit une chaîne de dépendances supplémentaire : la sélection ML lit les poids produits par `LogisticRegressionTrainer`, lesquels sont sauvegardés dans `ml_weights.txt` après l'entraînement supervisé. Le module d'apprentissage reste donc découplé du moteur de recherche : l'entraînement écrit un fichier de paramètres, et MCTS le charge uniquement lorsqu'on active ce mode de sélection.

3.4.8 Complexité de l'IA

Hypothèses et notations Dans cette section, nous analysons la complexité temporelle des méthodes principales des classes IA présentées en section 3.4. La variable n désigne le nombre de cases actives du plateau (typiquement 32, 50 ou 72), b représente le facteur de branchement moyen (nombre de coups légaux par position), et d la profondeur maximale de recherche. Nous distinguons la complexité *propre* d'une méthode de recherche (nombre de nœuds explorés) du coût réel, qui inclut aussi la génération des coups par le moteur via `getWhiteValidMoves()` et `getBlackValidMoves()`. Les résultats sont récapitulés dans le tableau 4, qui sert de référence de correspondance méthode par méthode.

Complexité des fonctions d'évaluation

SimpleEvaluator La méthode `evaluate(Board)` réalise un parcours linéaire de l'ensemble des cases actives, avec uniquement des tests bit-à-bit en temps constant à chaque itération. Sa complexité est donc $O(n)$. Cette analyse correspond à l'entrée `SimpleEvaluator::evaluate(Board)` du tableau 4.

AdvancedEvaluator La méthode `evaluate(Board)` suit le même schéma que `SimpleEvaluator` : un balayage linéaire des cases et un calcul de bonus d'avancement en temps constant par case. La complexité reste donc $O(n)$. Voir l'entrée `AdvancedEvaluator::evaluate(Board)` dans le tableau 4.

MaxEvaluator La méthode `evaluate(Board)` combine trois contributions : un parcours principal en $O(n)$, un test de centralité `isInCenter(...)` en $O(1)$ par case, puis une évaluation de mobilité dont le coût dépend des appels `getWhiteValidMoves()` et `getBlackValidMoves()`. Sa complexité globale s'écrit donc $O(n + M_w + M_b)$, où M_w et M_b représentent les coûts de génération des coups blancs et noirs. La méthode `isInCenter(int, int, int)` elle-même ne fait que des comparaisons arithmétiques constantes et s'exécute en $O(1)$. Ces deux résultats correspondent aux entrées `MaxEvaluator::evaluate(Board)` et `MaxEvaluator::isInCenter(...)` du tableau 4.

Complexité des algorithmes de recherche adversariale

MinMax La méthode `calculateBestMove(...)` lance une recherche complète jusqu'à `maxDepth` et hérite donc d'une complexité asymptotique en $O(b^d)$, hors coût exact de génération des coups. La méthode `findBestMoveAtDepth(...)` itère d'abord sur les coups racines puis délègue à l'exploration récursive `minMax / maxMin`, ce qui conduit au même ordre de grandeur. Plus finement, `minMax(...)` et `maxMin(...)` coûtent $O(b^k)$ à profondeur restante k , soit $O(b^d)$ au pire depuis la racine. Enfin, `checkTime(long)` effectue un test booléen constant en $O(1)$. Le détail de ces méthodes (`calculateBestMove`, `findBestMoveAtDepth`, `minMax`, `maxMin`, `checkTime`) est donné dans le tableau 4.

MinMaxAlphaBeta Les méthodes `calculateBestMove(...)` et `findBestMoveAtDepth(...)` suivent la même logique que `MinMax`, mais ajoutent les bornes α et β pour couper des sous-arbres. Ainsi, `maxValue(...)` et `minValue(...)` restent en $O(b^d)$ dans le pire cas (ordre des coups défavorable), mais peuvent atteindre $O(b^{d/2})$ dans le meilleur cas théorique avec un ordre des coups optimal. La méthode `checkTime(long)` reste en $O(1)$. Les bornes meilleur/pire cas et le détail par méthode (`findBestMoveAtDepth`, `maxValue`, `minValue`, `checkTime`) sont regroupés dans le tableau 4.

IterativeDeepening La méthode `calculateBestMove(...)` enchaîne des recherches successives de profondeur $1, 2, \dots, d$ tant que le budget temps n'est pas épuisé. Son coût total théorique est

$$\sum_{k=1}^d O(b^k) = O(b^d).$$

En pratique, le surcoût par rapport à une unique recherche à profondeur d est modéré, et il est compensé par la robustesse de la stratégie : en cas d'interruption temporelle, le meilleur coup d'une profondeur entièrement terminée est déjà disponible. Cette complexité correspond à l'entrée `IterativeDeepening::calculateBestMove(...)` du tableau 4.

Synthèse

TABLE 4 – Complexité détaillée, méthode par méthode, des classes IA

Classe	Méthode	Complexité	Indice dans le code
<code>SimpleEvaluator</code>	<code>evaluate(Board)</code>	$O(n)$	Une boucle unique parcourt les cases actives de 0 à <code>board.getIndexMax()-1</code> , avec tests de bits en temps constant.
<code>AdvancedEvaluator</code>	<code>evaluate(Board)</code>	$O(n)$	Même schéma que <code>SimpleEvaluator</code> : boucle linéaire + calcul arithmétique constant du bonus d'avancement.
<code>MaxEvaluator</code>	<code>evaluate(Board)</code>	$O(n + M_w + M_b)$	Boucle principale en n , appel de <code>isInCenter(...)</code> en temps constant, puis coût de mobilité via <code>getWhiteValidMoves()</code> et <code>getBlackValidMoves()</code> .
<code>MaxEvaluator</code>	<code>isInCenter(int, int, int)</code>	$O(1)$	Seulement des bornes calculées et des comparaisons.
<code>IterativeDeepening</code>	<code>calculateBestMove(...)</code>	$\sum_{k=1}^d O(b^k) = O(b^d)$	Boucle sur les profondeurs 1 à d , chaque itération appelant <code>MinMax</code> avec profondeur courante.
<code>MinMax</code>	<code>calculateBestMove(...)</code>	$O(b^d)$	Un appel principal à <code>findBestMoveAtDepth(...)</code> à profondeur <code>maxDepth</code> .
<code>MinMax</code>	<code>findBestMoveAtDepth(...)</code>	$O(b^d)$	Boucle sur les coups racines puis descente récursive dans l'arbre de jeu sans élagage.
<code>MinMax</code>	<code>minMax(...)</code>	$O(b^k)$	Pour profondeur restante k , exploration de tous les coups noirs puis appel récursif <code>maxMin(...)</code> .
<code>MinMax</code>	<code>maxMin(...)</code>	$O(b^k)$	Symétrique de <code>minMax(...)</code> pour les coups blancs, sans coupure.
<code>MinMax</code>	<code>checkTime(long)</code>	$O(1)$	Un seul test booléen via <code>isTimeExceeded(startTime)</code> .
<code>MinMaxAlphaBeta</code>	<code>calculateBestMove(...)</code>	Pire : $O(b^d)$	Lance <code>findBestMoveAtDepth(...)</code> ; l'élagage dépend de l'ordre des coups.
<code>MinMaxAlphaBeta</code>	<code>findBestMoveAtDepth(...)</code>	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Mise à jour de α et β à chaque enfant ; coupures quand $\beta \leq \alpha$.
<code>MinMaxAlphaBeta</code>	<code>maxValue(...)</code>	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Exploration récursive avec arrêt anticipé dès qu'une coupure $\beta \leq \alpha$ est détectée.
<code>MinMaxAlphaBeta</code>	<code>minValue(...)</code>	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Symétrique de <code>maxValue(...)</code> avec bornes inversées et mêmes règles de coupure.
<code>MinMaxAlphaBeta</code>	<code>checkTime(long)</code>	$O(1)$	Test constant, puis levée d'exception interne si le temps est dépassé.

4 Explication approfondie

4.1 F36 : Monte Carlo Tree Search (MCTS)

4.1.1 Logique théorique de l'algorithme

Contrairement à l'algorithme Minimax qui tente d'évaluer exhaustivement toutes les branches jusqu'à une profondeur donnée, le MCTS repose sur des simulations statistiques probabilistes. La recherche s'effectue en répétant une boucle de quatre phases fondamentales jusqu'à épuisement du temps de réflexion imparti :

Sélection À chaque itération, l'algorithme part de la racine de l'arbre (l'état actuel du damier) et descend vers les feuilles. Pour choisir quel chemin emprunter, il utilise la formule UCT (Upper Confidence Bound applied to Trees) :

$$UCT_j = \frac{w_j}{n_j} + C \sqrt{\frac{\ln n}{n_j}}$$

Où :

- w_j représente le nombre de victoires accumulées par les simulations passant par le nœud enfant j .
- n_j est le nombre total de simulations ayant traversé ce nœud j .
- n est le nombre total de visites du nœud parent.
- C est la constante d'exploration.

Concrètement : Cette formule résout le dilemme "Exploration vs Exploitation". Le terme de gauche favorise les coups ayant un fort taux de victoire (exploitation), tandis que le terme de droite grandit pour les coups peu testés (exploration). Si un nœud enfant n'a jamais été visité, sa valeur est considérée comme infinie pour forcer l'algorithme à tester chaque coup légal au moins une fois.

Expansion Une fois arrivé sur un nœud dont tous les coups possibles n'ont pas encore été explorés, l'algorithme choisit un de ces mouvements non testés, crée un nouveau nœud dans l'arbre pour représenter ce nouvel état, et passe à la phase suivante.

Simulation Depuis ce nœud nouvellement créé, l'algorithme lance une partie virtuelle. Il joue des coups totalement aléatoires pour les deux joueurs jusqu'à atteindre un état final (victoire, défaite ou égalité).

Rétropropagation Une fois la partie simulée terminée, le résultat (victoire, défaite ou nul) est remonté le long de l'arbre en suivant le chemin exact pris lors de la phase de sélection. Tous les nœuds ancêtres traversés voient leurs statistiques mises à jour : incrémentation de leur compteur de visites, et ajustement de leur score de victoire selon le camp gagnant.

4.1.2 Implémentation et optimisations dans notre projet

Notre classe `Mcts` gère cette recherche de manière optimisée en traduisant cette théorie à travers des choix d'architecture précis.

Traduction des phases en code La boucle principale est gérée par la méthode `calculateBestMove()`, qui s'exécute tant que la limite de temps (`isTimeExceeded()`) n'est pas atteinte.

Lors de la phase de **sélection**, la formule UCT est appliquée par `bestChildByUcb1()`. Si $n_j = 0$, la méthode renvoie `Double.MAX_VALUE` pour assurer l'exploration initiale :

```
double score = child.visits == 0
    ? Double.MAX_VALUE
    : child.winRate() + explorationConstant * Math.sqrt(logVisits / child.visits);
```

Lors de la **simulation** (`simulate()`), pour éviter des parties virtuelles infinies sur des situations bloquées, nous avons introduit une limite stricte de profondeur (`SIMULATION_DEPTH = 60`). Si cette limite est atteinte sans vainqueur, la partie rapporte un score de parité (`DRAW_SCORE = 0.5`), contre `WIN_SCORE = 1.0` pour une victoire.

Optimisation mémoire (ManagerUndoRedo) L'un des plus grands défis du MCTS est la gestion de la mémoire. Au lieu de cloner l'objet `Board` des milliers de fois pour chaque nœud et simulation (ce qui saturerait la RAM et ralentirait drastiquement l'exécution), nous utilisons une instance unique du plateau couplée à un `ManagerUndoRedo`. Chaque coup testé lors de la sélection, de l'expansion et de la simulation est appliqué physiquement sur le plateau, puis proprement annulé lors de la phase de `backpropagate` :

```
// Rétropropagation : on remonte jusqu'à la racine en annulant les coups joués
while (current != null) {
    current.visits++;
    // Mise à jour des scores...

    // Annulation physique du coup sur le plateau unique
    if (current.move != null) {
        undo.undo(current.player == PlayerColor.WHITE);
    }
    current = current.parent;
}
```

Guidage par Machine Learning

Pipeline d'entraînement du modèle Pour générer les poids (`mlWeights`) et le biais (`mlBias`) utilisés par cette fonction, nous avons implémenté un pipeline d'apprentissage supervisé autonome dans la classe `LogisticRegressionTrainer`. L'entraînement s'effectue via l'option en ligne de commande `-tr`. Le processus se divise en trois grandes étapes :

1. **Génération des données par Self-Play** : L'application fait s'affronter deux instances de l'IA (MCTS pur) l'une contre l'autre. Pour éviter les parties infinies, une limite stricte de 100 coups est fixée (`MAX_MOVES`). À chaque demi-coup, un vecteur de 6 caractéristiques (*features*) est extrait du plateau :

- Nombre de pions blancs et noirs.
- Nombre de dames blanches et noires.
- Différentiel de pions (Blancs - Noirs).
- Différentiel de dames (Blancs - Noirs).

Si la partie aboutit à une victoire, tous les vecteurs de cette partie sont étiquetés avec le résultat réel (1.0 pour une victoire des blancs, 0.0 pour les noirs). Les parties nulles sont ignorées pour ne pas bruite le modèle.

2. **Optimisation par descente de gradient** : Une fois les données collectées (par défaut sur 10 parties), la méthode `train()` ajuste les poids itérativement sur 1000 *epochs*. À chaque étape, pour chaque vecteur d'état, le modèle effectue une prédiction. L'erreur par rapport au résultat réel (*label*) est calculée, et les poids sont mis à jour selon un taux d'apprentissage (`learningRate = 0.01`) :

$$W_j = W_j - \alpha \times \text{erreur} \times X_j$$

Où α est le taux d'apprentissage, W_j le poids de la caractéristique j , et X_j la valeur de la caractéristique.

3. **Sauvegarde et chargement** : À l'issue de l'entraînement, les poids finaux et le biais sont sauvegardés dans un fichier texte (`ml_weights.txt`). Ce fichier est ensuite chargé dynamiquement par `Mcts` lors de son instantiation, garantissant une séparation propre entre la phase d'entraînement et la phase d'inférence (jeu).

Bilan de l'approche Bien évidemment, ce mode de sélection par apprentissage automatique n'a pas démontré de gain significatif en termes de performance ou de qualité de jeu par rapport à l'UCT classique. Cela s'explique par la simplicité de notre modèle : la régression logistique est un modèle linéaire, et nos 6 *features* mathématiques (uniquement basées sur le comptage matériel) ne capturent pas les subtilités positionnelles du jeu de dames.

Cependant, cette implémentation robuste démontre l'extensibilité de notre architecture. Le pipeline de génération de données, d'entraînement et d'inférence est pleinement fonctionnel, ce qui ouvre la porte à des améliorations futures plus sophistiquées (comme le remplacement de la régression logistique par un réseau de neurones profond évaluant des motifs complexes) pour renforcer drastiquement l'intelligence de notre agent.

Détermination du meilleur coup final À l'épuisement du temps de réflexion imparti, l'arbre cesse de croître. Quelle que soit l'heuristique de sélection utilisée durant la recherche (la formule UCT classique ou les prédictions du modèle de Machine Learning), la décision finale ne se base pas sur ces scores d'évaluation.

L'IA fait appel à la méthode `mostVisitedChild()`, qui sélectionne le nœud enfant de la racine possédant le **plus grand nombre de visites**. Ce critère est privilégié car il est statistiquement le plus robuste : une branche très visitée signifie qu'elle a continuellement été considérée comme la plus prometteuse par l'algorithme, et qu'elle a survécu à un grand nombre de simulations face aux meilleures réponses possibles de l'adversaire.

Stratégie de validation et de tests Afin de garantir la robustesse de notre implémentation du MCTS, nous avons développé une série de tests unitaires centralisés dans la classe `MctsTest`. La première étape de cette validation consiste à vérifier l'intégrité de l'arbre de recherche et de son cycle de vie. Cela passe notamment par le contrôle de l'initialisation des paramètres fondamentaux, comme l'affectation correcte de la constante d'exploration à sa valeur théorique par défaut de $\sqrt{2}$. Nous nous assurons également de la stabilité de l'algorithme face aux cas limites. Par exemple, si l'IA est sollicitée sur un plateau où aucun mouvement légal n'est possible (en toute fin de partie), la méthode de recherche est tenue de retourner une absence de coup sans provoquer la moindre erreur d'exécution.

```
@DisplayName("Default constructor sets exploration constant to sqrt(2)")
void testDefaultConstructorExplorationConstant() {
    Mcts mcts = new Mcts();
    assertEquals(Mcts.DEFAULT_EXPLORATION, mcts.explorationConstant, 1e-9);
}
```

Le mode de sélection par apprentissage automatique (`SelectionMode.ML`) a nécessité une approche de test plus pointue. Étant donné que cette fonctionnalité manipule des classes internes privées, comme les nœuds de l'arbre, et lit des fichiers de poids externes, nous avons fait appel à la réflexion Java pour isoler et éprouver ces composants. Cette technique nous permet de confirmer que la fonction de chargement analyse correctement les vecteurs de caractéristiques, tout en gérant proprement les fichiers corrompus via des exceptions contrôlées. De plus, la réflexion nous sert à simuler artificiellement des nœuds en cours de partie pour vérifier que l'algorithme bascule bien sur le moteur d'inférence ML, au lieu de la formule UCT, lorsque l'option correspondante est activée au lancement.

```
@DisplayName("Test direct du switch case ML dans select")
void testSelectOnlyMl() throws Exception {
    mcts.setSelectionMode(SelectionMode.ML);
    Object result = selectMethod.invoke(mcts, root, board, undo);
    assertEquals(child, result, "Le switch doit choisir l'enfant via le mode ML");
}
```

Enfin, nous contrôlons rigoureusement la cohérence mathématique des calculs qui guident les choix de l'IA. Nos tests s'assurent par exemple que l'évaluation par régression logistique génère systématiquement une probabilité valide, strictement bornée entre 0 et 1, grâce à l'application de la fonction sigmoïde sur les statistiques du plateau. Du côté de l'approche statistique classique, nous validons le mécanisme de convergence de l'UCT en vérifiant que le code attribue bien la priorité maximale (`Double.MAX_VALUE`) aux branches non explorées. C'est cette garantie mathématique qui force l'algorithme à évaluer chaque coup légal au moins une fois avant de se concentrer sur l'exploitation des meilleures variantes.

```
@DisplayName("evaluateMl correctly calculates probability with loaded weights")
void testEvaluateMlCalculation(@TempDir Path tempDir) throws Exception {
    double probability = (double) evaluateMlMethod.invoke(mcts, childNode, board, undo);
    assertTrue(probability >= 0.0 && probability <= 1.0);
}
```

5 Revue critique de projet

5.1 Bug

5.2 Axes d'amélioration

Conclusion

Références

- [1] Brandon Corfman. Raven Checkers — open-source checkers with ai. <https://github.com/bcorfman/raven-checkers>, 2023. Consulté le 8 avril 2026.
- [2] mortexsa. AwsDame — multiplayer checkers platform. <https://github.com/mortexsa/AwsDame>, 2023. Consulté le 8 avril 2026.
- [3] Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson, 4 edition, 2020.

A Liste des 40 besoins

TABLE 5 – État d'avancement des besoins fonctionnels

ID	Description du besoin	Statut
F1	Lancement du programme	
F2	Fichier de configuration	
F3	Internationalisation (i18n)	
F4	Interface en ligne de commande	
F5	Mode blitz	
F6	Mode contest	
F7	Interface graphique	
F8	Mode IA	
F9	Taille du damier	
F10	Gestion des règles	
F11	Gestion des erreurs de l'utilisateur	
F12	Gestion du undo/redo	
F13	Gestion des conseils	
F14	Gestion du blitz	
F15	Shell interactif	

Suite à la page suivante...

ID	Description du besoin	Statut
F16	Proposer de sauvegarder avant de quitter	
F17	Édition de la commande en cours	
F18	Historique des commandes tapées	
F19	Complétion des commandes	
F20	Notation des coups	
F21	Sauvegarde et restauration d'une partie	
F22	Commentaires dans la sauvegarde	
F23	Format des paramètres du jeu	
F24	Format du damier	
F25	Format de l'historique	
F26	Module bitboard	
F27	Algorithmes de manipulation des bitboards	
F28	Menus de l'application	
F29	Raccourcis clavier	
F30	Drag'n'Drop	
F31	Temps de réflexion borné	
F32	Minimax avec $\alpha\beta$ -pruning	
F33	Fonctions d'évaluation pour Minimax	
F34	Approfondissement itératif de Minimax	
F35	Profondeur de recherche de Minimax	
F36	Monte Carlo Tree Search (MCTS)	
F37	Fonction de sélection par Machine Learning	
F38	Découverte de serveurs et connexion client-serveur basique	
F39	Serveur multi-clients et gestion de parties	
F40	Système d'invitations et gestion avancée des joueurs	